COP2800C Module 8 Graded Programming Assignment – Using Packages

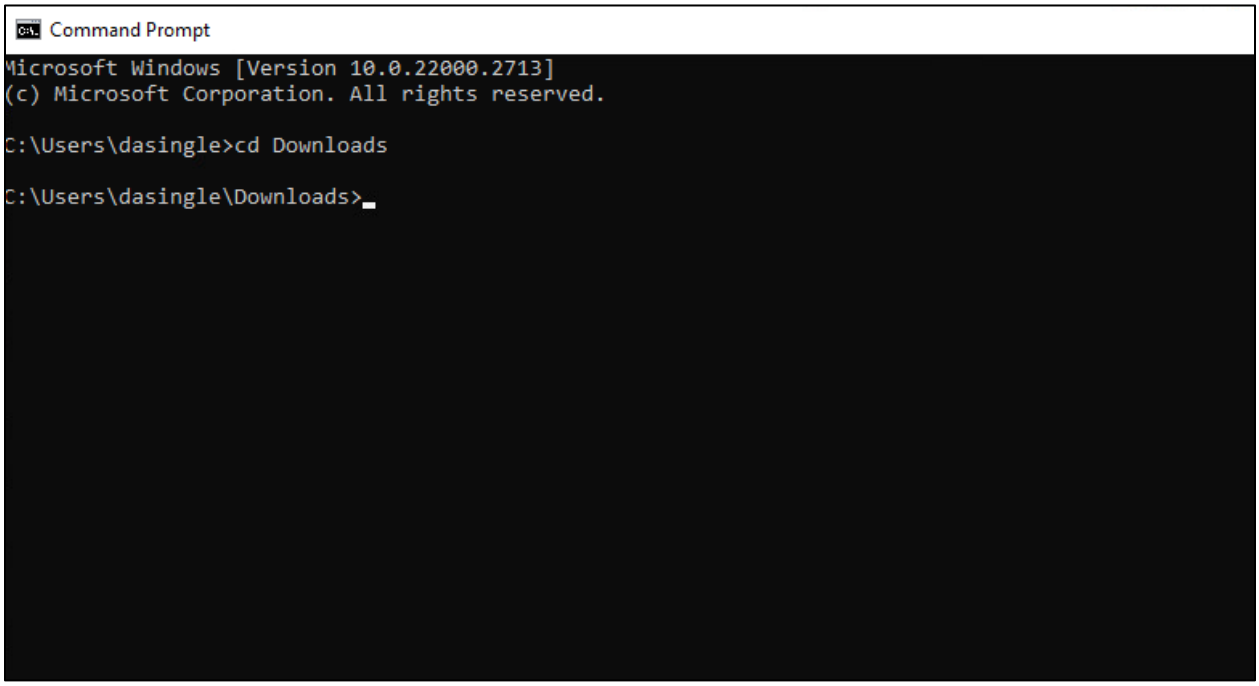
In this assignment, we will implement a package for all our Java source code files. This will cause our build environment to expand into a folder structure. Packages are useful for projects that use multiple files, and the packaging structure is typically used by professional IDEs as a consistent way to organize a project. You will need to use the Git command-line tool to complete this assignment so that your solution is submitted with the proper folder structure.

NOTE: Git is preinstalled on the Horizon Academic IT W11 system).

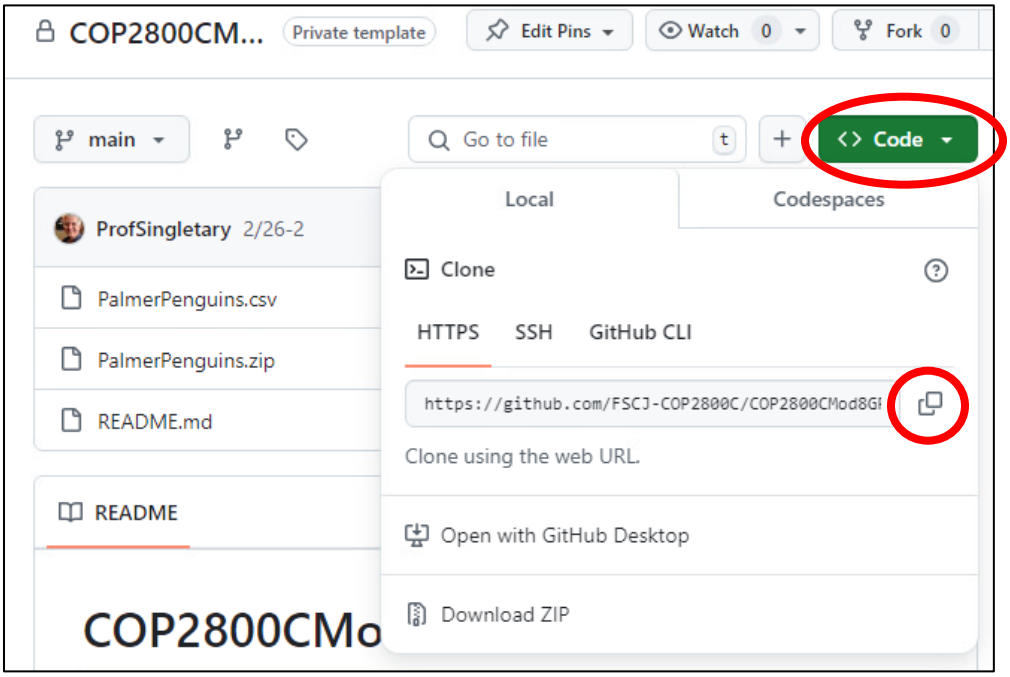
NOTE2: Submitting your solution correctly using GIT with the specified package structure is required for you to receive credit for this assignment.

Instructions (these instructions were executed on the Horizon Academic IT W11 system).

- After accepting the GitHub classroom invitation, clone the starter repository on the local system using these steps:
  - Open a Command Prompt tool by searching for **cmd** in the Windows search bar.
  - Navigate to your Documents or Downloads folder using **cd**:

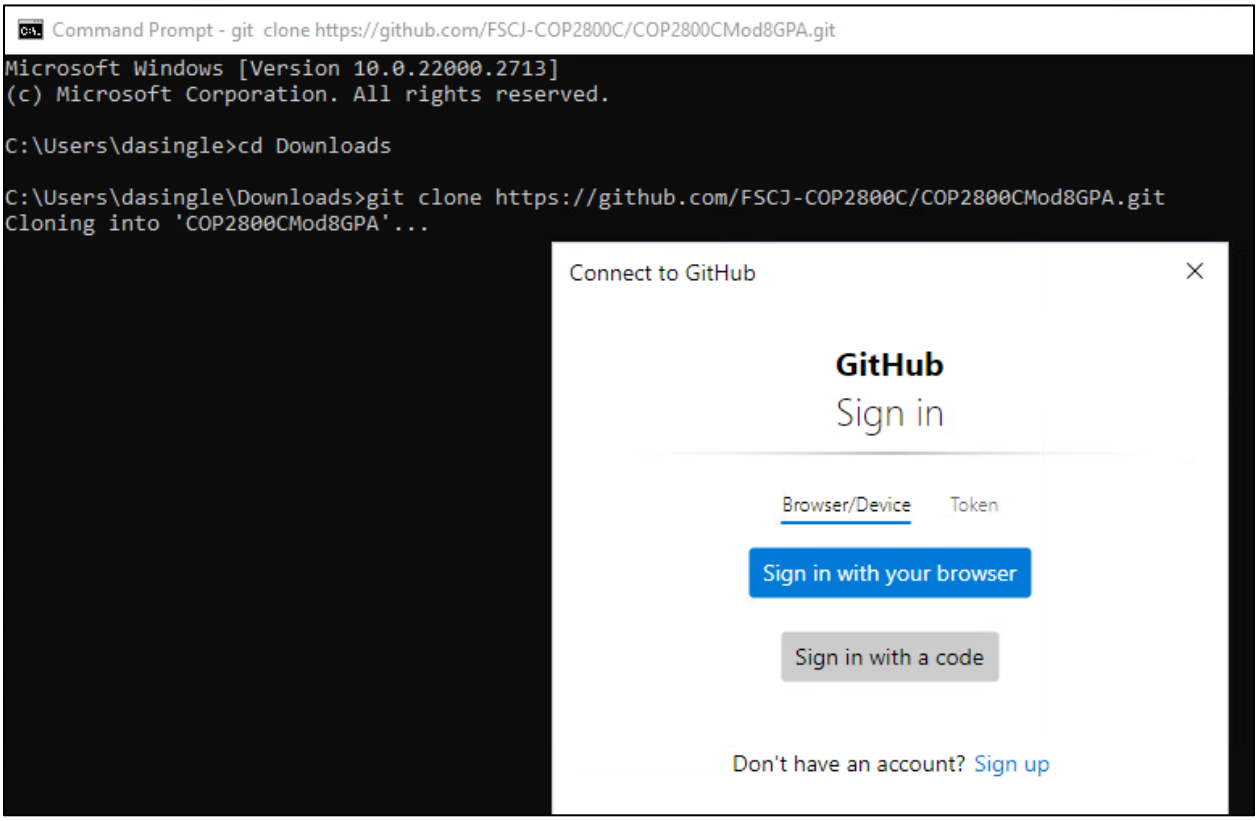


- Log in to GitHub and access your personal repository, then open the **Code** drop-down and click the copy icon:

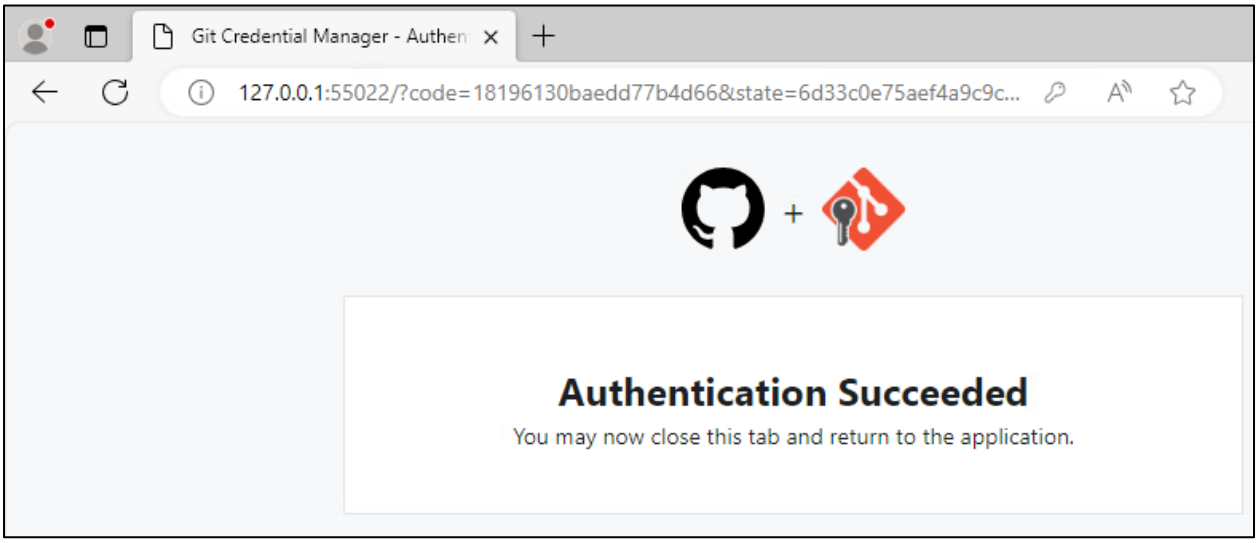


- The URL is now copied to your clipboard. Back in the Command Prompt window, enter the **git clone** command and paste the URL after it:

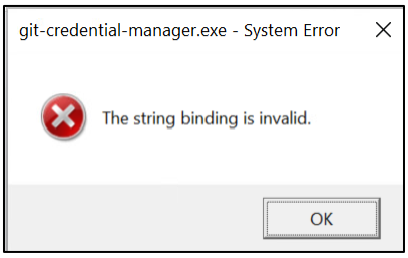
- At this point you may be prompted to authenticate to GitHub by logging in with a browser (Horizon will bring up Edge for this).



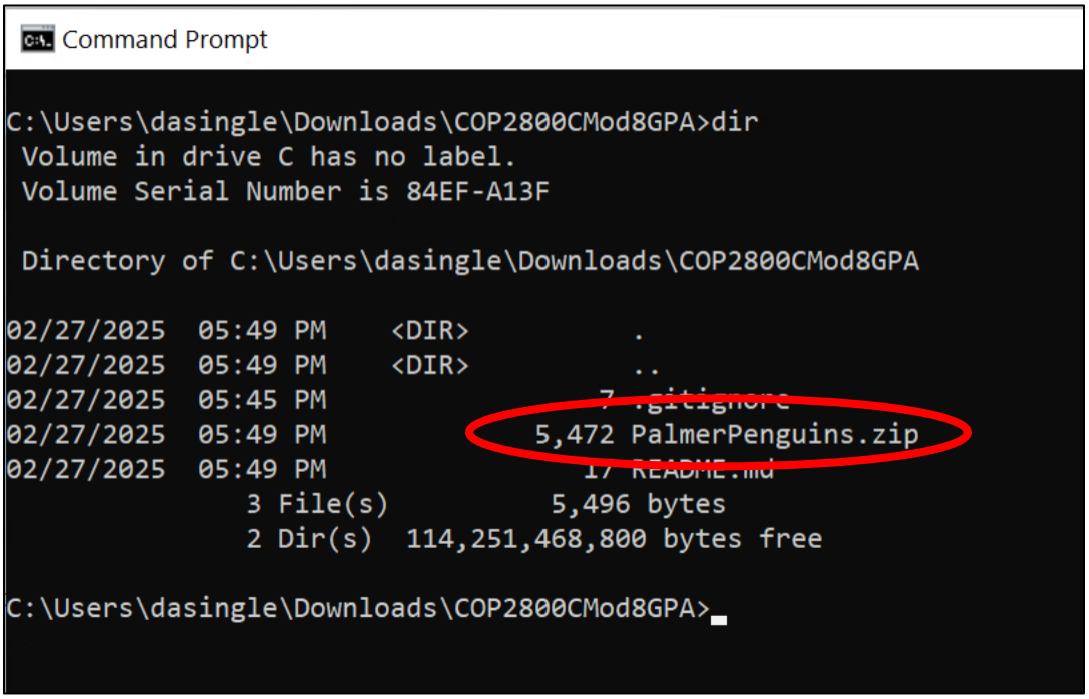
- After authenticating you can dismiss the browser window:



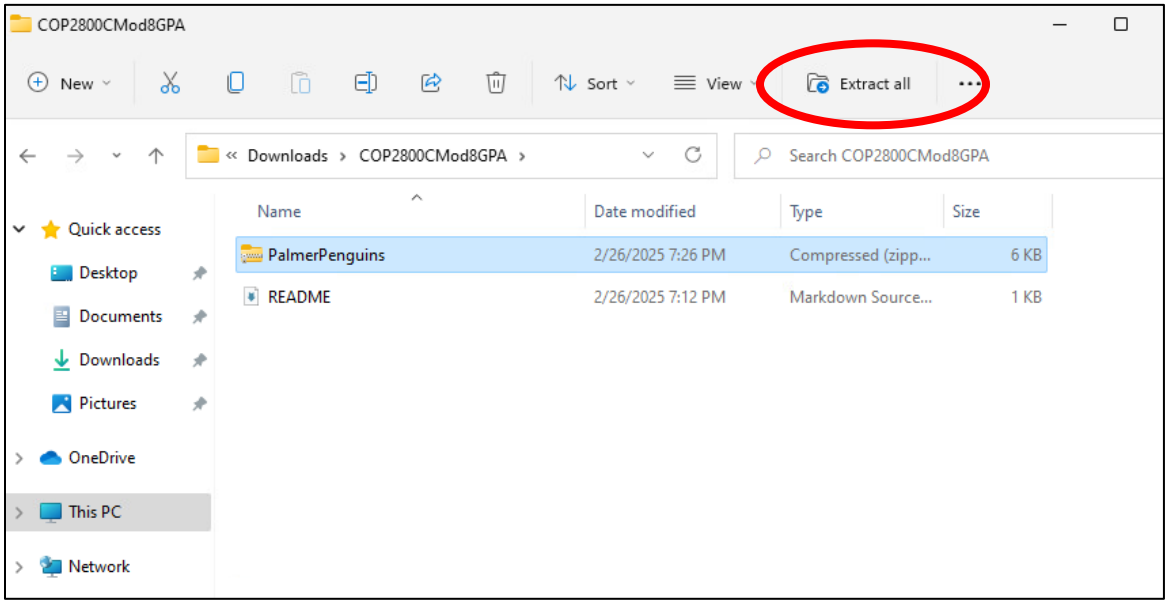
- If you encounter this error dialog after authenticating it can be ignored



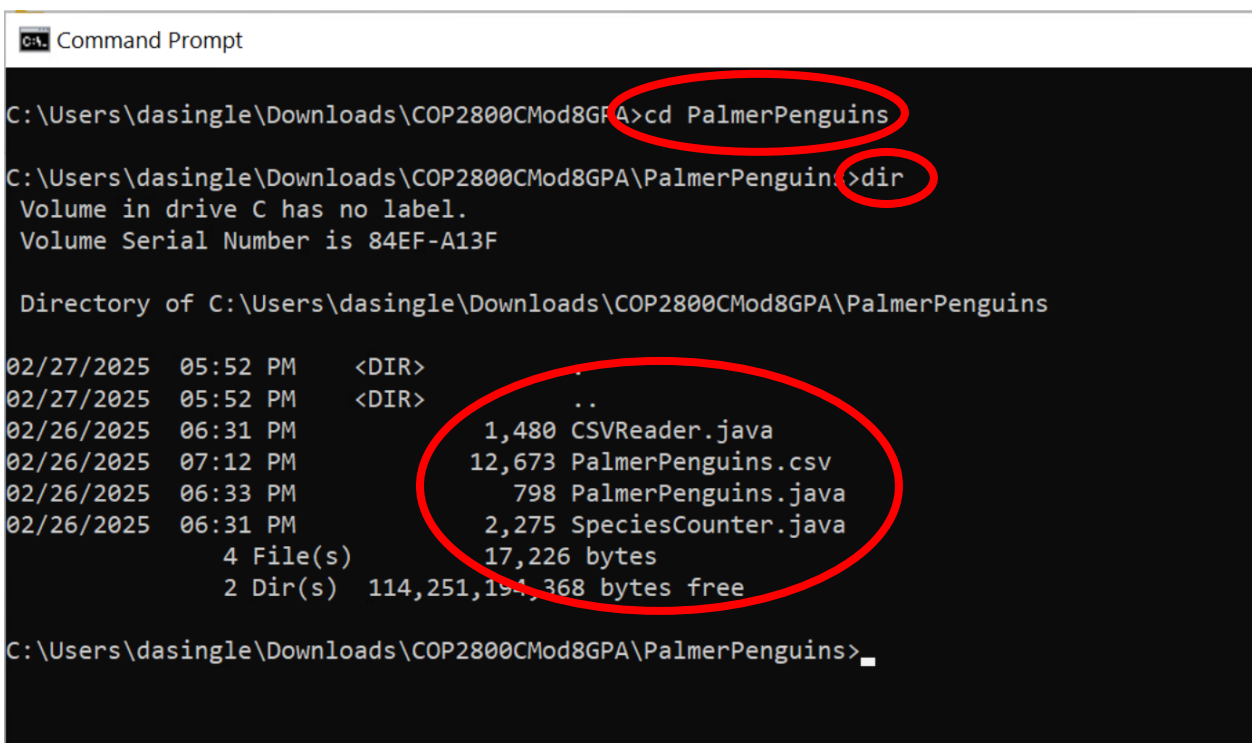
- Navigate into the repo folder (your folder name will not be the same as mine) using the **cd** command, then run a **dir** command in the Command Prompt window and you will see a .zip file; this file contains the Java source code files for the Module 7 solution and the PalmerPenguins CSV file.



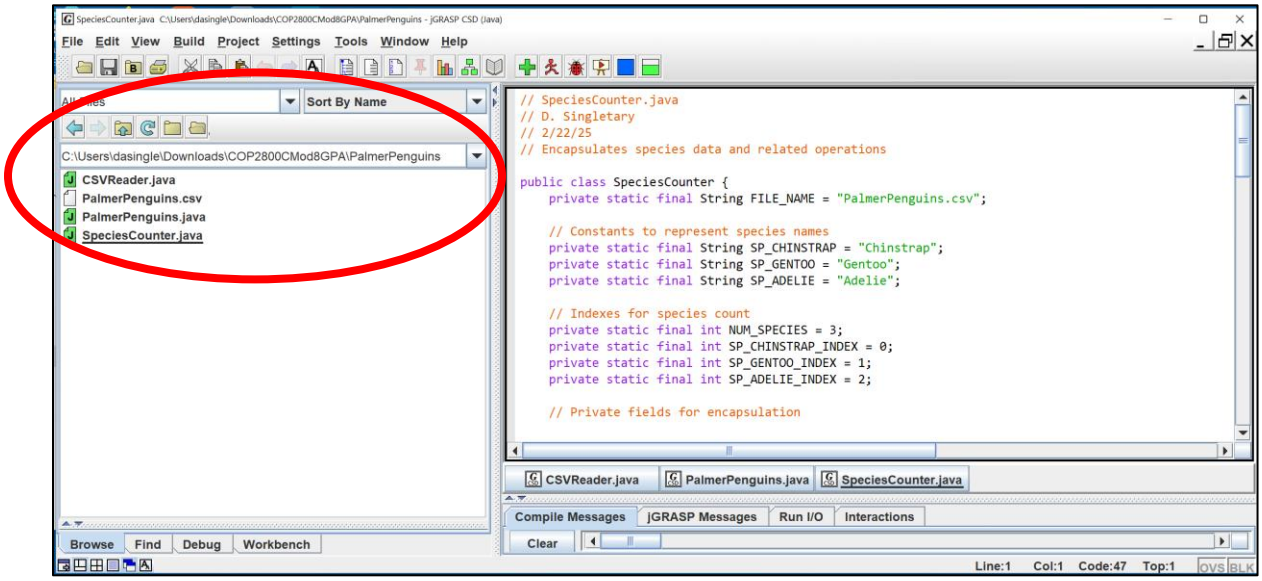
- Use Windows File Explorer to extract the PalmerPenguins.zip folder in the starter repo. This will expand the file into a folder named PalmerPenguins which contains the files described above. This will be our project folder.



- Navigate into the PalmerPenguins project folder using the **cd** command and run **dir** to view the files:



- Open the files in jGrasp. Verify that you are in the PalmerPenguins project folder in the jGrasp file browser pane; if not, click on the folders as necessary to navigate to that folder (you may need to use the up arrow to move up and back down). You will be able to see all of the files if you are in the correct location.



- Add the following package statement to all three Java files. This statement must be inserted after any ID headers and before any other content (including import statements) in the files. Include a vertical space (blank line) between the last line of the ID header and your package statement. Include another vertical space after the package statement.

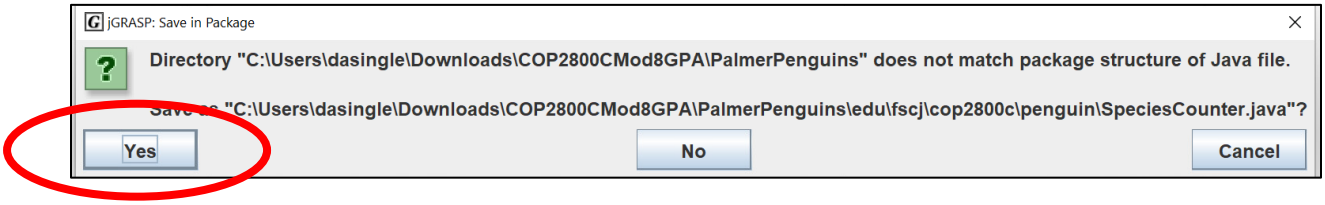
```
// ID Header lines
// ...

package edu. fscj.cop2800c.penguin;
```

```java
// PalmerPenguins.java
// D. Singletary
// 2/22/25
// Refactored Palmer Penguins analysis using object-oriented approach

package edu.fscj.cop2800c.penguin;

public class PalmerPenguins {
    public static void main(String[] args) {
        // Create an instance of SpeciesCounter
        SpeciesCounter counter = new SpeciesCounter();

        // Read species data
        counter.readSpeciesData();

        // Initialize species count
        counter.initializeSpeciesCount();

        // Check if data is empty
        if (counter.isDataEmpty()) {
```

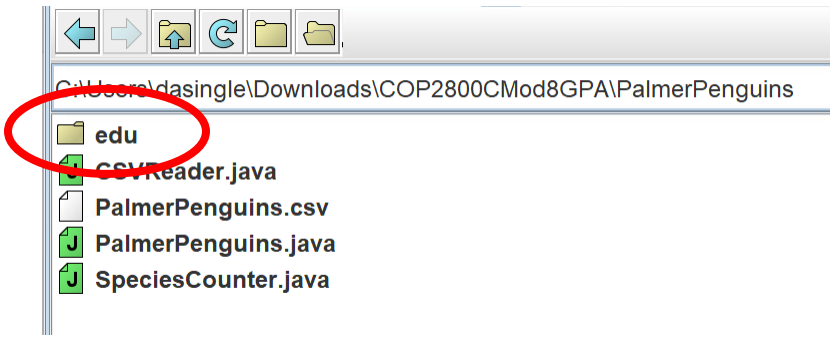
```java
// SpeciesCounter.java
// D. Singletary
// 2/22/25
// Encapsulates species data and related operations

package edu.fscj.cop2800c.penguin;

public class SpeciesCounter {
    private static final String FILE_NAME = "PalmerPenguins.csv";

    // Constants to represent species names
    private static final String SP_CHINSTRAP = "Chinstrap";
    private static final String SP_GENTOO = "Gentoo";
    private static final String SP_ADELIE = "Adelie";
```

```java
// CSVReader.java
// D. Singletary
// 2/22/25
// Reads a CSV file line-by-line

package edu.fscj.cop2800c.penguin;

import java.io.*;
import java.util.*;

public class CSVReader {
    /**
     * Reads a file line by line, extracts the specified column from
     * each line, and returns an array containing the extracted values.
     *
```

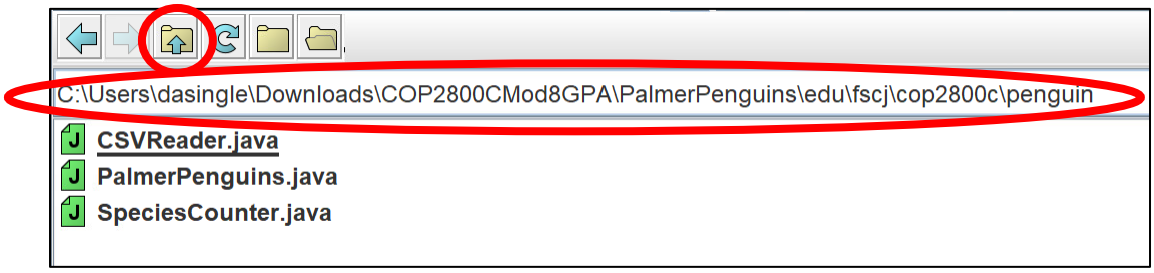
- Use the "Save As" option in jGrasp under the File menu to save each file. jGrasp will prompt that the file is not in the correct location and will prompt you to save it in the proper folder; answer "Yes" to this. Be sure to do this for all three .java files.



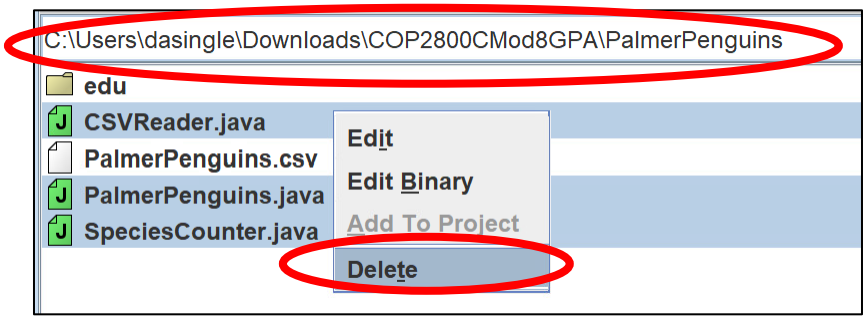
- Notice that when you save the files in their package location, jGrasp will create the folder structure. Now at the project folder level in the jGrasp file browser you have an "edu" folder, which is the top level of the package.



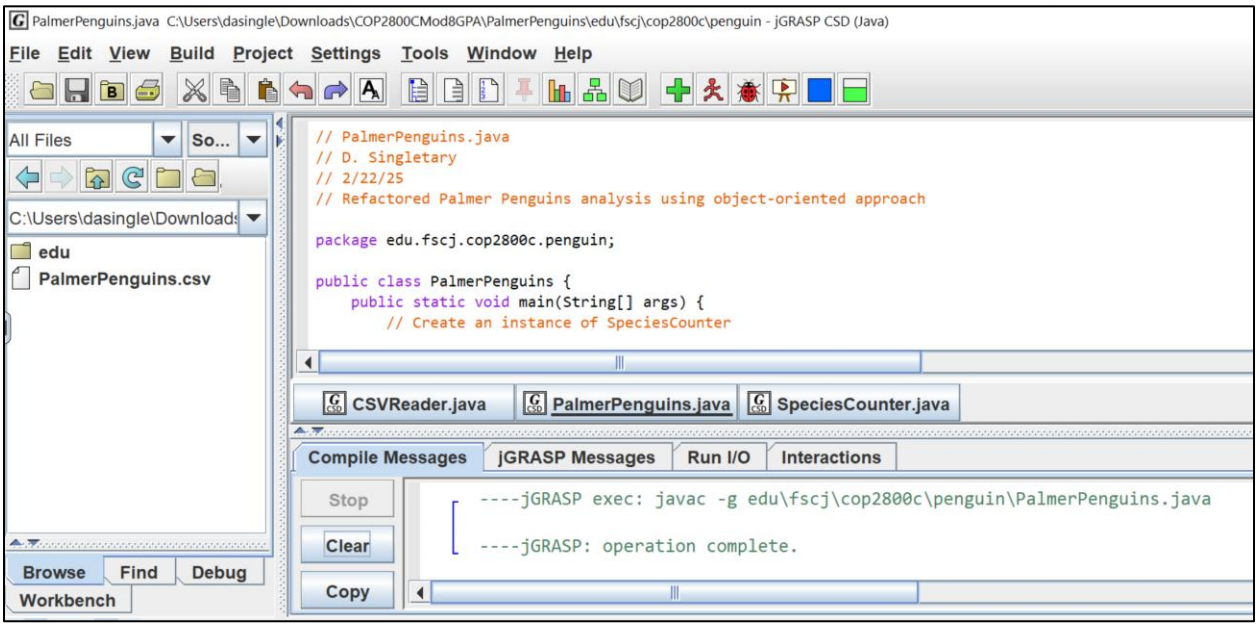
- If you drill down on this folder by double-clicking on it in the file browser you will see that you can move all the way down into the "penguin" sub-package, where the source files now reside. Be sure you use the up arrow to navigate back up to the PalmerPenguins project folder before proceeding.



- At this point we no longer need the source files at the top level of the project folder, delete them so they are not pushed to the repo when we are done with the project. Be sure you are in the top level project folder before doing this. You can use Ctrl-Right Click to select only the .java files, then right-click to delete them.



- Be sure to keep the source files open; you actually only need to keep the PalmerPenguins.java file open since building it will build all of the dependent classes. Build this file, then execute the program to verify it works correctly.



- After executing the program to verify correct operation, return back to the Command Prompt window and run **git add .** (don't forget the dot at the end). This will stage your files so they can be pushed to GitHub.

```
Command Prompt

C:\Users\dasingle\Downloads\COP2800CMod8GPA>git add .
```

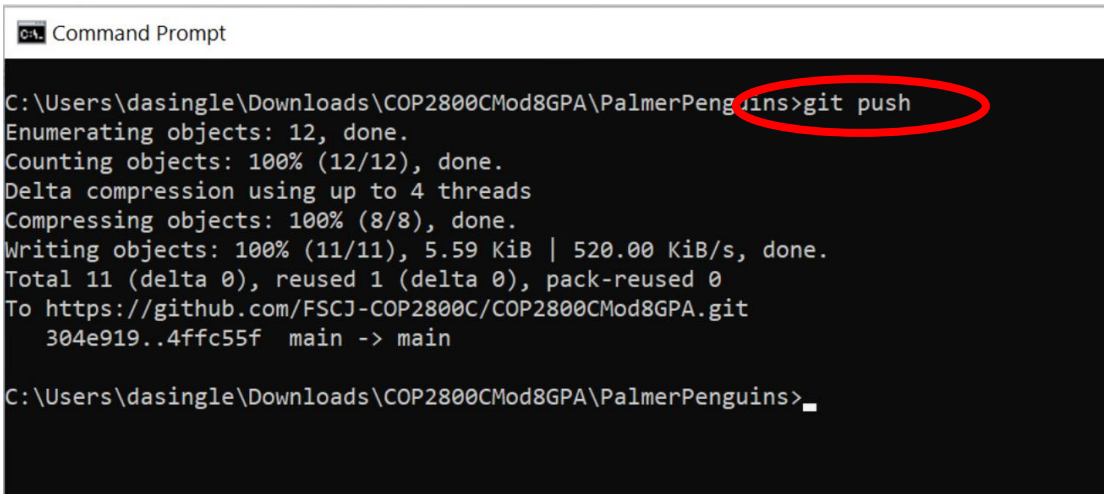
- Run **git status** to view the files that will be commited (in green text).

```
Command Prompt

C:\Users\dasingle\Downloads\COP2800CMod8GPA\PalmerPenguins>git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   PalmerPenguins.csv
        new file:   edu/fscj/cop2800c/penguin/CSVReader.java
        new file:   edu/fscj/cop2800c/penguin/PalmerPenguins.java
        new file:   edu/fscj/cop2800c/penguin/SpeciesCounter.java


C:\Users\dasingle\Downloads\COP2800CMod8GPA\PalmerPenguins>
```

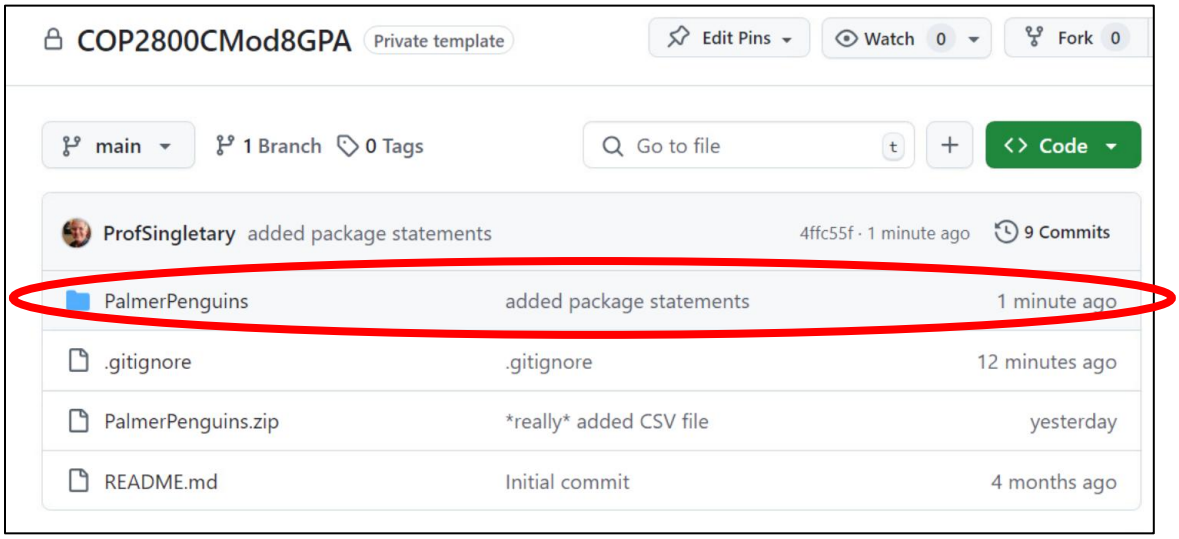
- Now run a git commit command to provide a message that will be associated with this check-in. The command must include the "-m" option and a quoted message as shown in this window, e.g. **git commit -m "added package statements"**

```
Command Prompt

C:\Users\dasingle\Downloads\COP2800CMod8GPA\PalmerPenguins>git commit -m "added package statements"
[main 4ffc55f] added package statements
 Committer: Singletary <david.singletary@fscj.edu>
Your name and email address were configured automatically based
on your username and hostname. Please check that they are accurate.
You can suppress this message by setting them explicitly. Run the
following command and follow the instructions in your editor to edit
your configuration file:

    git config --global --edit

After doing this, you may fix the identity used for this commit with:

    git commit --amend --reset-author

 4 files changed, 485 insertions(+)
 create mode 100644 PalmerPenguins/PalmerPenguins.csv
 create mode 100644 PalmerPenguins/edu/fscj/cop2800c/penguin/CSVReader.java
 create mode 100644 PalmerPenguins/edu/fscj/cop2800c/penguin/PalmerPenguins.java
 create mode 100644 PalmerPenguins/edu/fscj/cop2800c/penguin/SpeciesCounter.java

C:\Users\dasingle\Downloads\COP2800CMod8GPA\PalmerPenguins>
```

- Next we will push the code up to GitHub using **git push**:



- Once this is done you can refresh your repo page and see the commit message and new folder:



Submission

Your project submission to the repo is all you need. The package structure must be properly committed, there should be no .java files at the top level of your project folder.